TaskForge — Day 4 Notes

Day 4: Modular Architecture, Service Layer & Transaction Safety

Goal: Refactor the TaskForge backend so that application setup, HTTP routes, request/response validation, business logic, and database operations have clear responsibilities.

Day 4 outcome: Created separate Pydantic schemas, added response models, moved job endpoints into an `APIRouter`, introduced a service layer, and added rollback handling for failed database transactions.

What changed today?

Before	After
All endpoints and database logic in one `main.py`	`main.py` registers routers; `jobs.py` handles HTTP routes
Routes directly query and update SQLAlchemy	`job_service.py` owns job database operations
Request validation only	Request and response schemas
Commit failures had no explicit rollback	Write operations roll back and re-raise on errors

The work builds on Day 3: PostgreSQL persistence with FastAPI and SQLAlchemy. The existing API behavior and response shapes were kept while the internal structure was improved.

---

1. Separate request and response schemas

Earlier, `JobCreate` and `JobStatusUpdate` were defined in `main.py`. On Day 4, these Pydantic models were moved into `app/schemas.py`. This keeps data validation definitions separate from application startup and route code.

File: `app/schemas.py`

```
from typing import Any, Literal
from pydantic import BaseModel, Field, ConfigDict


class JobCreate(BaseModel):
    type: str = Field(min_length=1)
    payload: dict[str, Any]


class JobStatusUpdate(BaseModel):
    status: Literal[
        "queued", "processing", "completed", "failed"
    ]


class JobResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: str
    type: str
    payload: dict[str, Any]
    status: str


class JobCreateResponse(BaseModel):
```

```
    message: str
    job: JobResponse


class JobListResponse(BaseModel):
    count: int
    skip: int
    limit: int
    jobs: list[JobResponse]


class JobUpdateResponse(BaseModel):
    message: str
    job: JobResponse
```

Key concepts

- **Request schema:** validates data sent by the client before the endpoint uses it.
- **Response schema:** describes and validates the structure returned by an endpoint.
- `Literal`: restricts `status` to the listed values; an unsupported status is rejected by validation.
- `Field(min_length=1)`: requires the job type string to contain at least one character.
- `ConfigDict(from_attributes=True)`: allows Pydantic v2 to read values from object attributes, such as a SQLAlchemy ORM object.

The response wrapper schemas preserve the existing API shape: create and update responses contain a `message` and a nested `job`; the list response contains `count`, `skip`, `limit`, and `jobs`.

---

2. Add response_model to endpoints

FastAPI's `response_model` tells the framework which schema describes an endpoint's output. FastAPI validates and serializes the returned data against that schema and uses it in the generated OpenAPI documentation.

Applied response models

Endpoint	Response schema
`POST /api/v1/jobs`	`JobCreateResponse`
`GET /api/v1/jobs`	`JobListResponse`
`PATCH /api/v1/jobs/{job_id}/status`	`JobUpdateResponse`
`GET /api/v1/jobs/{job_id}`	Kept its existing response shape; no wrapper schema was added in this step

Important: `response_model` does not create, query, or save database records. SQLAlchemy still performs database work; `response_model` validates the shape of the data returned to the client.

---

3. Move job endpoints into an APIRouter

As an application grows, keeping every endpoint in `main.py` becomes difficult to maintain. `APIRouter` groups related endpoints so they can live in separate modules.

Project structure after route separation

```
app/
├── routes/
│   ├── __init__.py
│   └── jobs.py
├── __init__.py
├── database.py
├── main.py
├── models.py
└── schemas.py
```

In app/routes/jobs.py

```
from fastapi import APIRouter

router = APIRouter()


@router.post("/api/v1/jobs")
def create_job_route(...):
    ...
```

In app/main.py

```
from fastapi import FastAPI
from .routes import jobs

app = FastAPI(title="TaskForge API")
app.include_router(jobs.router)


@app.get("/")
def home():
    return {"message": "Welcome to TaskForge API"}


@app.get("/health")
def health_check():
    return {"status": "healthy"}
```

- `APIRouter()`: creates a router that can hold related endpoints.
- `@router.get()`, `@router.post()`, `@router.patch()`: register endpoints on that router.
- `include_router()`: attaches the router to the main FastAPI application.
- `main.py`: becomes a small application entry point rather than a file containing every endpoint.

---

4. Introduce the service layer

After route separation, `jobs.py` still contained database queries and writes. The service layer moves those operations into a dedicated module. The route handles HTTP concerns; the service handles the job operation.

Project structure

```
app/
├── routes/
│   ├── __init__.py
│   └── jobs.py
├── services/
│   ├── __init__.py
│   └── job_service.py
├── database.py
├── main.py
├── models.py
└── schemas.py
```

File: `app/services/job_service.py` — service functions

```
from sqlalchemy.orm import Session
from ..models import Job


def create_job(db: Session, job_type: str, payload: dict):
    db_job = Job(type=job_type, payload=payload)
    db.add(db_job)
    db.commit()
    db.refresh(db_job)
    return db_job


def get_all_jobs(db: Session, skip: int, limit: int):
    total = db.query(Job).count()
    jobs = (
        db.query(Job)
        .order_by(Job.id)
        .offset(skip)
        .limit(limit)
        .all()
    )
    return total, jobs


def get_job_by_id(db: Session, job_id: str):
    return db.query(Job).filter(Job.id == job_id).first()


def update_job_status(db: Session, job_id: str, new_status: str):
    job = db.query(Job).filter(Job.id == job_id).first()
    if job is None:
        return None
    job.status = new_status
    db.commit()
    db.refresh(job)
    return job
```

What each service does

Service function	Responsibility
`create_job()`	Creates and saves a job, then returns it.
`get_all_jobs()`	Counts jobs and retrieves a paginated set.
`get_job_by_id()`	Finds one job; returns `None` if it does not exist.
`update_job_status()`	Finds a job, updates its status, and saves it.

A service function should not need to know about HTTP status codes or FastAPI's `HTTPException`. For example, `get_job_by_id()` returns `None` when no record exists; the route converts that result into an HTTP 404 response.

---

5. Connect routes to services

The route imports the service functions and calls them. It no longer writes SQLAlchemy queries for each operation.

Example: create route

```
from ..services.job_service import create_job


@router.post(
    "/api/v1/jobs",
    status_code=status.HTTP_202_ACCEPTED,
    response_model=JobCreateResponse,
)
def create_job_route(
    job_data: JobCreate,
    db: Session = Depends(get_db),
):
    db_job = create_job(
        db,
        job_data.type,
        job_data.payload,
    )

    return {
        "message": "Job created successfully",
        "job": serialize_job(db_job),
    }
```

The route still handles request validation, HTTP response formatting, and status codes. The service handles the database operation. The same pattern was applied to listing, retrieving, and updating jobs.

Why serialize_job() remains in the route module

```
def serialize_job(job: Job) -> dict[str, Any]:
    return {
        "id": job.id,
        "type": job.type,
        "payload": job.payload,
        "status": job.status,
    }
```

This helper converts the SQLAlchemy `Job` object into the dictionary used in the API response. It needs the `Job` model imported in `jobs.py` because `Job` is used in the function's type annotation.

---

6. Add transaction safety with rollback

A database write can fail during commit. If a transaction fails, SQLAlchemy requires a rollback before the session can be used for further database work. The service functions now catch the exception, roll back, and re-raise the original error.

Create operation with rollback

```
def create_job(db: Session, job_type: str, payload: dict):
    try:
        db_job = Job(type=job_type, payload=payload)
        db.add(db_job)
        db.commit()
        db.refresh(db_job)
        return db_job

    except Exception:
        db.rollback()
        raise
```

Status update with rollback

```
def update_job_status(db: Session, job_id: str, new_status: str):
    job = db.query(Job).filter(Job.id == job_id).first()

    if job is None:
        return None

    try:
        job.status = new_status
        db.commit()
        db.refresh(job)
        return job

    except Exception:
        db.rollback()
        raise
```

Understand the error-handling statements

- `try:` runs the database write operation.
- `except Exception:` catches an exception raised during the operation.
- `db.rollback()`: rolls back the failed transaction so the session is not left in a failed transaction state.
- `raise`: re-raises the original exception instead of hiding it.

The rollback pattern was added to the create and update write operations. Read-only service functions do not need this write-transaction rollback wrapper. The database session dependency still closes the session after the request.

Engineering note: Catching `Exception` is broad. In production systems, consider logging the error and handling expected database exceptions deliberately. Do not return raw database error details to API clients.

---

7. Verification and testing completed

The project was checked after the refactor using Python's compile check and the interactive Swagger UI.

Syntax/import checks

```
python -m py_compile app/schemas.py
python -m py_compile app/main.py
python -m py_compile app/routes/jobs.py app/services/job_service.py app/main.py
```

Manual API checks in Swagger

Endpoint	Expected behavior	Verified
`POST /api/v1/jobs`	Create job; HTTP 202; initial status `queued`	Yes
`GET /api/v1/jobs`	Return paginated jobs; HTTP 200	Yes
`GET /api/v1/jobs/{job_id}`	Return requested job; HTTP 200	Yes
`PATCH /api/v1/jobs/{job_id}/status`	Update status; HTTP 200	Yes
Nonexistent job ID	Return HTTP 404	Yes

Swagger showed all four job routes after `APIRouter` was connected. The user also tested create, list, retrieve, update, and the missing-job case after the service-layer integration.

---

8. Save Day 4 work with Git

The Day 4 changes were committed locally with the message shown below. A push was initially rejected because GitHub had an additional commit deleting the Day 1 notes PDF.

```
git add .
git commit -m "Complete Day 4: modular architecture and service layer"
```

The local and remote branches had diverged from the same parent. The safe resolution was to merge the remote-tracking branch, review/accept Git's merge commit message in Vim using `Esc`, `:wq`, `Enter`, and then push.

```
git fetch origin
git log --oneline --graph --decorate --all -10
git merge origin/main
git push origin main
```

Important: Do not use `git push --force` for this situation. A normal merge preserves both the local Day 4 commit and the remote deletion commit. Confirm the push succeeds before considering the work backed up on GitHub.

---

9. Day 4 summary and next steps

- Pydantic schemas now live in `schemas.py`.
- Response models describe and validate the output of selected endpoints.
- Job endpoints are grouped in `routes/jobs.py` using `APIRouter`.
- `main.py` registers the router and keeps the home and health endpoints.
- `job_service.py` contains the create, list, retrieve, and update database operations.
- Routes translate service results into HTTP responses, including 404 for missing jobs.
- Create and update operations roll back failed transactions and re-raise exceptions.
- All four job endpoints were tested in Swagger after the refactor.

Day 5 recommended focus — automated testing

Learn pytest, FastAPI `TestClient`, test fixtures, isolated test databases, and tests for successful and failing API requests. The next goal is to prove TaskForge works automatically, not only through manual Swagger testing.

Core lesson: A maintainable backend separates responsibilities. Routes handle HTTP, services handle business operations, schemas validate data, models define database tables, and the database stores persistent records.